ARRAYS

Syntax · Examples · Patterns

LEVEL-WISE COMPLETE GUIDE

Beginner → Intermediate → Advanced → Expert

Java | Python | C++ | DSA | Interview Ready

What is an Array? • Core Concepts

An **array** is a contiguous block of memory that stores elements of the same data type. It provides O(1) random access via index, making it one of the most fundamental data structures.

0	1	2	3	4	5
10	20	30	40	50	60
0x064	0x068	0x06C	0x070	0x074	0x078

↑ Index ↑ Value ↑ Memory Address (each int = 4 bytes)

Operation	Time	Space
Access by index	O(1)	O(1)
Search (unsorted)	O(n)	O(1)
Insert at end	O(1) amortized	O(1)
Insert at middle	O(n)	O(1)
Delete at end	O(1)	O(1)
Delete at middle	O(n)	O(1)

Level 1 — Beginner

1.1 Declaration & Initialization

```
// ■■ Java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
int[] arr = new int[5];
int[] arr = {10, 20, 30, 40, 50};
int[][] matrix = new int[3][3];
# ■■ Python ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
arr = [10, 20, 30, 40, 50]
arr = [0] * 5
matrix = [[0]*3 for _ in range(3)]
// ■■ C++ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
int arr[5] = {10, 20, 30, 40, 50};
vector<int> v = {10, 20, 30, 40, 50};
```

1.2 Access, Update, Length

```
// Java
int x = arr[2]; // access index 2 → 30
arr[2] = 99; // update
int len = arr.length; // length
# Python
x = arr[2] # 30
arr[2] = 99
length = len(arr)
last = arr[-1] # negative indexing
```

1.3 Traversal

```
// Java — for loop
for (int i = 0; i < arr.length; i++) {
System.out.println(arr[i]);
}
// Java — enhanced for
for (int x : arr) System.out.println(x);
# Python — enumerate
for i, val in enumerate(arr):
print(i, val)
```

1.4 Linear Search

```
// Java
int linearSearch(int[] arr, int target) {
for (int i = 0; i < arr.length; i++)
if (arr[i] == target) return i;
return -1;
}
# Python
def linear_search(arr, target):
for i, val in enumerate(arr):
if val == target: return i
return -1
```

1.5 Reverse an Array

```
// Java — two pointer
void reverse(int[] arr) {
int l = 0, r = arr.length - 1;
while (l < r) {
int tmp = arr[l]; arr[l] = arr[r]; arr[r] = tmp;
l++; r--;
}
}
# Python one-liner
arr[::-1] # reversed copy
arr.reverse() # in-place
```

Level 2 — Intermediate

2.1 Binary Search (sorted array)

Complexity Time: O(log n) | Space: O(1)

```
// Java iterative
int binarySearch(int[] arr, int target) {
int lo = 0, hi = arr.length - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2;
if (arr[mid] == target) return mid;
else if (arr[mid] < target) lo = mid + 1;
else hi = mid - 1;
}
return -1;
}
# Python
import bisect
idx = bisect.bisect_left(arr, target) # stdlib
```

2.2 Prefix Sum

Use-case Range sum queries in O(1) after O(n) preprocessing

```
// Java
int[] prefix = new int[n + 1];
for (int i = 0; i < n; i++)
prefix[i + 1] = prefix[i] + arr[i];
// Sum of arr[l..r] (inclusive, 0-indexed)
int rangeSum = prefix[r + 1] - prefix[l];
# Python
prefix = [0] * (len(arr) + 1)
for i, v in enumerate(arr):
prefix[i+1] = prefix[i] + v
range_sum = prefix[r+1] - prefix[l]
```

2.3 Sliding Window (fixed size)

Use-case	Max/min/sum of every subarray of size k

```
// Max sum subarray of size k — Java
int maxSum(int[] arr, int k) {
int window = 0;
for (int i = 0; i < k; i++) window += arr[i];
int best = window;
for (int i = k; i < arr.length; i++) {
window += arr[i] - arr[i - k];
best = Math.max(best, window);
}
return best;
}
```

2.4 Two Pointer — Pair with Target Sum

```
// Java (sorted array required)
boolean hasPair(int[] arr, int target) {
int l = 0, r = arr.length - 1;
while (l < r) {
int sum = arr[l] + arr[r];
if (sum == target) return true;
else if (sum < target) l++;
else r--;
}
return false;
}
```

2.5 Sort Algorithms on Arrays

```
// Java built-in
Arrays.sort(arr); // O(n log n)
Arrays.sort(arr, Collections.reverseOrder()); // descending (Integer[])
# Python
arr.sort() # in-place
sorted_arr = sorted(arr, reverse=True) # new list
arr.sort(key=lambda x: -x) # custom key
```

Level 3 — Advanced

3.1 Kadane's Algorithm — Max Subarray Sum

Complexity	Time: O(n) | Space: O(1)

```
// Java
int maxSubarraySum(int[] arr) {
int maxSoFar = arr[0], maxEndingHere = arr[0];
for (int i = 1; i < arr.length; i++) {
maxEndingHere = Math.max(arr[i], maxEndingHere + arr[i]);
maxSoFar = Math.max(maxSoFar, maxEndingHere);
}
return maxSoFar;
}
# Python
def max_subarray(arr):
best = cur = arr[0]
for x in arr[1:]:
cur = max(x, cur + x)
best = max(best, cur)
return best
```

3.2 Dutch National Flag (3-way partition)

Use-case	Sort array of 0s, 1s, 2s in one pass — O(n) time, O(1) space

```
// Java
void sortColors(int[] arr) {
int lo = 0, mid = 0, hi = arr.length - 1;
while (mid <= hi) {
if (arr[mid] == 0) swap(arr, lo++, mid++);
else if (arr[mid] == 1) mid++;
else swap(arr, mid, hi--);
}
}
```

3.3 Merge Intervals

```java
// Java
int[][] merge(int[][] intervals) {
Arrays.sort(intervals, (a,b) -> a[0] - b[0]);
List<int[]> res = new ArrayList<>();
for (int[] iv : intervals) {
if (res.isEmpty() || res.get(res.size()-1)[1] < iv[0])
res.add(iv);
else
res.get(res.size()-1)[1] = Math.max(res.get(res.size()-1)[1], iv[1]);
}
return res.toArray(new int[0][]);
}
```

3.4 Next Greater Element (Monotonic Stack)

Complexity	Time: O(n) | Space: O(n)

```java
// Java
int[] nextGreater(int[] arr) {
int n = arr.length;
int[] res = new int[n];
Arrays.fill(res, -1);
Deque<Integer> stack = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
while (!stack.isEmpty() && arr[stack.peek()] < arr[i])
res[stack.pop()] = arr[i];
stack.push(i);
}
return res;
}
```

Level 4 — Expert

4.1 Trapping Rainwater

Classic | Hard — Two pointer approach O(n) time, O(1) space

```
// Java — two pointer O(n) O(1)
int trap(int[] height) {
int l = 0, r = height.length - 1;
int leftMax = 0, rightMax = 0, water = 0;
while (l < r) {
if (height[l] <= height[r]) {
if (height[l] >= leftMax) leftMax = height[l];
else water += leftMax - height[l];
l++;
} else {
if (height[r] >= rightMax) rightMax = height[r];
else water += rightMax - height[r];
r--;
}
}
return water;
}
```

4.2 Product of Array Except Self

Constraint	No division allowed — O(n) time, O(1) extra space

```
// Java
int[] productExceptSelf(int[] nums) {
int n = nums.length;
int[] out = new int[n];
out[0] = 1;
for (int i = 1; i < n; i++) // left prefix
out[i] = out[i-1] * nums[i-1];
int right = 1;
for (int i = n-1; i >= 0; i--) { // right suffix
out[i] *= right;
right *= nums[i];
}
return out;
}
```

4.3 Rotate Matrix 90°

```
// Java — transpose then reverse rows
void rotate(int[][] m) {
int n = m.length;
// Transpose
for (int i = 0; i < n; i++)
for (int j = i+1; j < n; j++) {
int tmp = m[i][j]; m[i][j] = m[j][i]; m[j][i] = tmp;
}
// Reverse each row
for (int[] row : m) {
int l = 0, r = n - 1;
while (l < r) { int t = row[l]; row[l++] = row[r]; row[r--] = t; }
}
}
```

4.4 Sparse Table (Range Minimum Query)

Complexity	Build: O(n log n) | Query: O(1) — immutable arrays only

```java
// Java
int LOG = 17;
int[][] sp = new int[LOG][n];
sp[0] = arr.clone();
for (int j = 1; j < LOG; j++)
for (int i = 0; i + (1<<j) <= n; i++)
sp[j][i] = Math.min(sp[j-1][i], sp[j-1][i + (1<<(j-1))]);
// Query min in [l, r]
int query(int l, int r) {
int k = Integer.numberOfTrailingZeros(Integer.highestOneBit(r-l+1));
return Math.min(sp[k][l], sp[k][r-(1<<k)+1]);
}
```

Patterns Quick Reference

Two Pointers	l/r moving inward — pair sum, container with most water, palindrome check
Sliding Window	Fixed or variable window — max sum, longest subarray, min window substring
Prefix Sum	Precompute cumulative sums — range queries, subarray sum equals k
Monotonic Stack	Decreasing/increasing stack — next greater, largest rect in histogram
Binary Search	On sorted arrays — search, lower/upper bound, rotated search
Divide & Conquer	Merge sort, quicksort, maximum subarray (modified Kadane)
Greedy on Array	Activity selection, gas station, jump game I & II
DP on Array	LIS, LCS, coin change, house robber, buy-sell stock

Common Interview Questions by Level

Problem	Technique	Difficulty
Find max/min in array	Linear scan	■ Easy
Two Sum	Hash Map / Two Pointer	■ Easy
Best Time to Buy/Sell Stock	Greedy (one pass)	■ Easy
Contains Duplicate	Hash Set	■ Easy
Maximum Subarray	Kadane's	■ Medium
Product Except Self	Prefix/Suffix product	■ Medium
3Sum	Sort + Two Pointer	■ Medium
Container With Most Water	Two Pointer	■ Medium
Longest Subarray with Sum k	Prefix Sum + Hash Map	■ Medium
Trapping Rainwater	Two Pointer / Stack	■ Hard
Median of Two Sorted Arrays	Binary Search	■ Hard

Largest Rectangle in Histogram	Monotonic Stack	■ Hard